

VLSI II Final Project Report

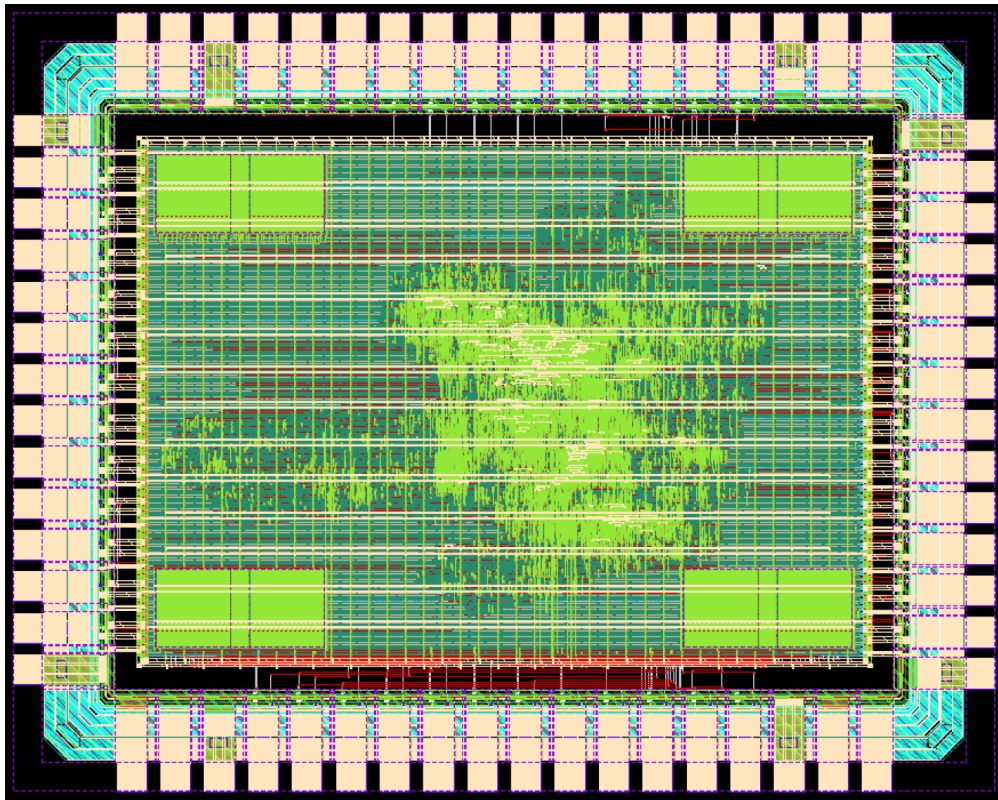
Extension of Croc SoC with an FIR Filter Accelerator

*Chip Name: **CrocFIRe***

Tejeswar Anand Ramachandran Vikash Kalyani Sankararaman
vlsi2.29fs26

ETH Zürich
June 30, 2026

Abstract. This report documents the design, integration, and physical implementation of a hardware-accelerated 32-tap FIR filter on the Croc SoC, developed as part of the VLSI II course at ETH Zürich. The baseline design was extended by integrating a parameterizable FIR filter accelerator into the user domain, supporting 1, 2, 4, 8 and 16 parallel MAC units. The accelerator implements a blocked FIR architecture operating on signed 8-bit signal samples with Q0.7 fixed-point coefficients, communicating with the Croc using the Open Bus Interface (OBI). The report covers the RTL design, verification, testing, and the backend physical implementation, culminating in a fully placed and routed GDSII layout.



Contents

1	Introduction	3
2	Design	3
2.1	Architecture Overview	3
2.2	Main State Machine	4
2.3	FIR Filter Accelerator Datapath	5
2.4	FIFO State Machine	6
2.5	FIFO Datapath	7
2.6	Memory-Mapped Registers	7
2.7	Software Driver and Accelerator Programming	8
3	Backend Implementation	8
3.1	Floorplanning and SRAM Macro Placement	8
3.2	Placement and Design Rule Check Fixes	8
3.3	Design Rule Violations	8
3.4	Area Analysis	9
3.5	DRC and LVS Checks	9
4	Testing and Verification	10
4.1	System Constraints and Limitations	10
4.1.1	Design Constraints	10
4.1.2	Batch Processing Limitation	10
4.2	Functional Verification	10
4.2.1	Testing the Parameterized MAC Datapath	10
4.2.2	Testing the System - Parameterized MAC Datapath + FIFO	10
4.2.3	Coefficient Edge Cases	11
4.2.4	Signal Sample Edge Cases	11
5	Performance Analysis	11
5.1	Accelerator Speedup Analysis	11
5.2	Performance-Area Tradeoff	11
5.3	Power Efficiency Analysis	12
6	Conclusion	14

1 Introduction

Finite Impulse Response (FIR) filters are fundamental components of digital signal processing systems and are widely used for applications including audio processing, communication systems, biomedical signal conditioning, and sensor data filtering. Each output sample requires a sequence of multiply-accumulate (MAC) operations across all filter taps, making FIR filtering computationally intensive and inefficient when executed purely in software.

The objective of this project was to implement a hardware accelerator for a 32-tap FIR filter capable of significantly reducing the execution time of the filtering operation. The accelerator processes signed 8-bit input samples using Q0.7 fixed-point filter coefficients and supports a parameterizable number of parallel MAC units. Configurations with 1, 2, 4, 8, and 16 MAC units were implemented to investigate the trade-off between hardware resources, and throughput.

In addition to improving execution performance, the parameterizable architecture enables a systematic comparison of different design points in terms of performance, power and silicon area. This report describes the accelerator architecture, its integration into the existing system, and the evaluation of the resulting implementations across the supported hardware configurations.

2 Design

2.1 Architecture Overview

Figure 1 shows the updated Croc SoC architecture with the FIR accelerator integrated into the user domain. The accelerator interfaces with the rest of the SoC through two OBI ports. The subordinate (SBR) OBI port allows the CVE2 core to configure the accelerator by writing to its memory-mapped registers, including the number of input samples, filter coefficients, and the start signal, and to poll the done status register upon completion. The manager (MGR) OBI port allows the accelerator to autonomously fetch signal samples from SRAM Bank 2 and write computed results to SRAM Bank 3, without CPU involvement after initial configuration. Within the user domain, an address decoder distributes incoming transactions to the FIR accelerator, the User ROM, or a default error subordinate for unmapped accesses.

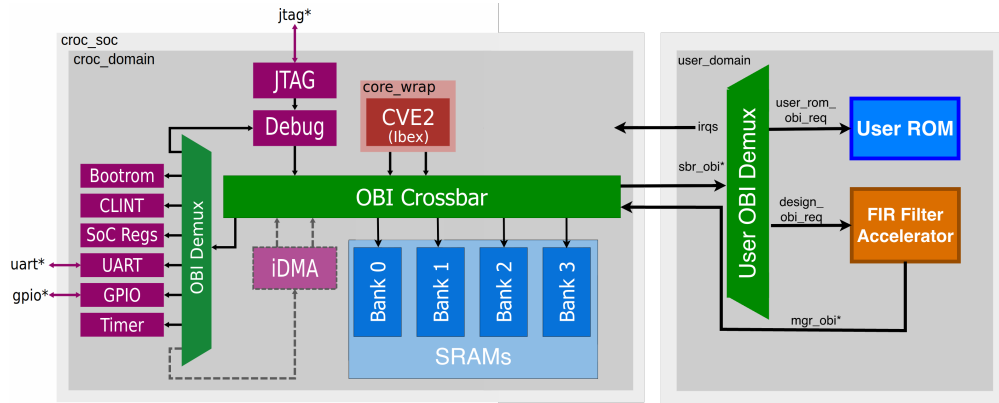


Figure 1: Block diagram of the modified Croc SoC with the FIR filter accelerator and User ROM integrated into the user domain

Table 1 presents the complete SoC memory map including both the original Croc peripherals and the additions made in this project. Two additional SRAM banks were added to support the accelerator: Bank 2 at 0x10001000 to 0x10001800 for input signal samples and Bank 3 at 0x10001800 to 0x10002000 for computed output results. The FIR accelerator’s Memory-Mapped (MM) registers are accessible at 0x20000400, and the User ROM at 0x20000000.

Start Address	Stop Address	Description
0x0000_0000	0x0004_0000	Debug module (JTAG)
0x0200_0000	0x0200_4000	BootROM
0x0204_0000	0x0208_0000	CLINT peripheral
0x0300_0000	0x0300_1000	SoC control/info registers
0x0300_2000	0x0300_3000	UART peripheral
0x0300_5000	0x0300_6000	GPIO peripheral
0x0300_A000	0x0300_B000	Timer peripheral
0x0300_B000	0x0300_C000	(optional) DMA configuration
0x1000_0000	0x1000_0800	SRAM Bank 0
0x1000_0800	0x1000_1000	SRAM Bank 1
0x1000_1000	0x1000_1800	SRAM Bank 2 (For storing signal samples)
0x1000_1800	0x1000_2000	SRAM Bank 3 (For storing FIR filtered results)
0x2000_0000	0x8000_0000	Passthrough to User Domain
0x2000_0000	0x2000_0400	Reserved for User ROM text
0x2000_0400	0x2000_0490	FIR Accelerator MM Registers

Table 1: SoC Memory Map

2.2 Main State Machine

Figure 2 shows the main state machine of the FIR Accelerator. The main FSM has four states - `IDLE`, `READ_DATA`, `COMPUTE` and `COMPUTE_COMPLETE`.

In `IDLE`, the accelerator waits for the CPU to assert the start bit via the control register. Before proceeding, it validates that the configured sample count lies within the supported range of 36 to 540. If the value is out of range, an error flag is set and the accelerator remains in the `IDLE` state. A minimum of 36 samples is required because the filter needs at least 32 samples to fill the initial window, plus one complete prefetch word of 4 samples to begin the sliding-window computation.

In `READ_DATA`, the accelerator sequentially reads 32-bit words from the Data SRAM. Since each word carries four packed 8-bit samples, eight reads are sufficient to fill the 32-entry shift register. A ninth read fills the prefetch buffer which holds four samples, supplying the next input sample one at a time during computation. Once these nine reads complete, the accelerator transitions to `COMPUTE`.

In `COMPUTE`, the accelerator produces one output sample every $32/\text{NumMacs}$ computation cycles (e.g., 4 cycles for 8 MACs), with NumMacs MAC units operating in parallel to accumulate partial products before generating a result. Each cycle, the filter coefficients are applied to the current 32-sample window, and when a result is ready it is pushed into the Output FIFO. The filter window shifts forward by one position, evicting the oldest entry and loading the next sample from the prefetch buffer. Once the prefetch buffer is exhausted, a new SRAM read is issued to refill it. This refill happens concurrently with ongoing computation, ensuring the datapath is not stalled waiting for memory. Once all expected output samples have been produced, the accelerator moves to `COMPUTE_COMPLETE`.

In `COMPUTE_COMPLETE`, all output samples have been produced and computation is disabled. The accelerator waits for the output FIFO to drain completely to SRAM Bank 3. Once the FIFO is empty, the `done_check` status register is set, which the CPU detects by polling and the accelerator returns to `IDLE`. The FIFO writeback mechanism is handled by a secondary concurrent FSM, described in Section 2.4.

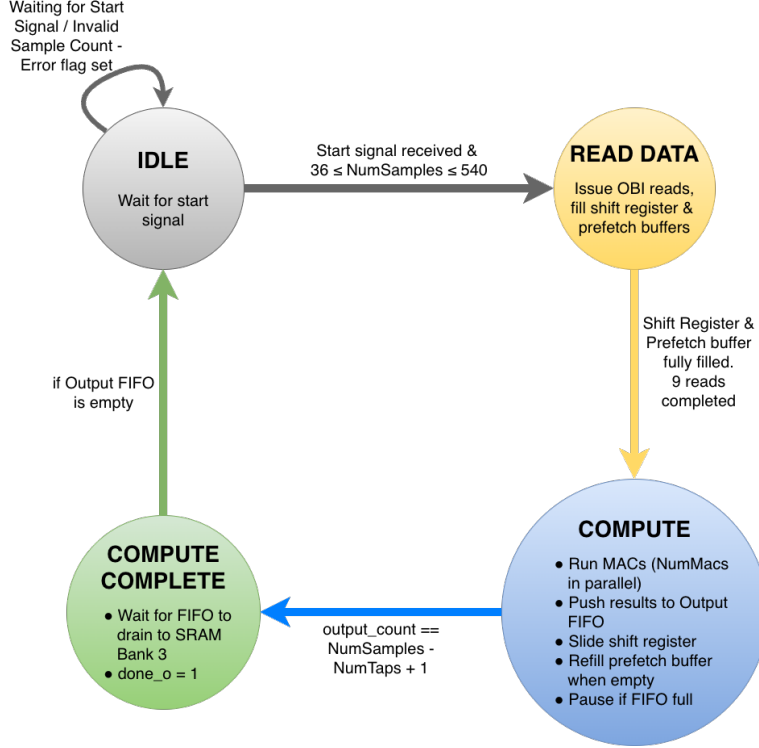


Figure 2: State diagram of the FIR accelerator's Main FSM

2.3 FIR Filter Accelerator Datapath

The filter datapath maintains two internal data stores as shown in Figure 3. The filter window is a 32-entry shift register holding the current block of input samples against which the coefficients are applied. The coefficients are stored in a 32-entry 8-bit signed register array. The prefetch buffer is a four-entry register that holds the next four samples, not yet introduced into the filter window.

During initial loading, each incoming 32-bit SRAM word is unpacked into four 8-bit values and written sequentially into the filter window. An `initial_load_done` flag tracks when the filter window is full, after which subsequent SRAM words are directed to the prefetch buffer instead.

Computation is performed using NumMacs parallel MAC units. With 32 taps and NumMacs MAC units, each output is produced in $32/\text{NumMacs}$ clock cycles. Figure 3 shows the datapath for the 8 MAC configuration, where each output sample is produced in 4 clock cycles. All partial sums are accumulated in a 21-bit accumulator, sized to prevent overflow in the worst case: 32 products of two signed 8-bit values yields a maximum magnitude of $32 \times 127 \times 127 = 516,128$, which fits within a 21-bit two's complement range. Once all 32 taps have been processed, the result is right-shifted by 7 bits to rescale it, consistent with the Q0.7 fixed-point representation used for the filter coefficients. The scaled output is sign-extended to 32 bits and forwarded to the output FIFO.

After each output, the filter window slides forward by one position: the oldest sample is evicted, all remaining samples shift down, and the next byte from the prefetch buffer is loaded in at the far end.

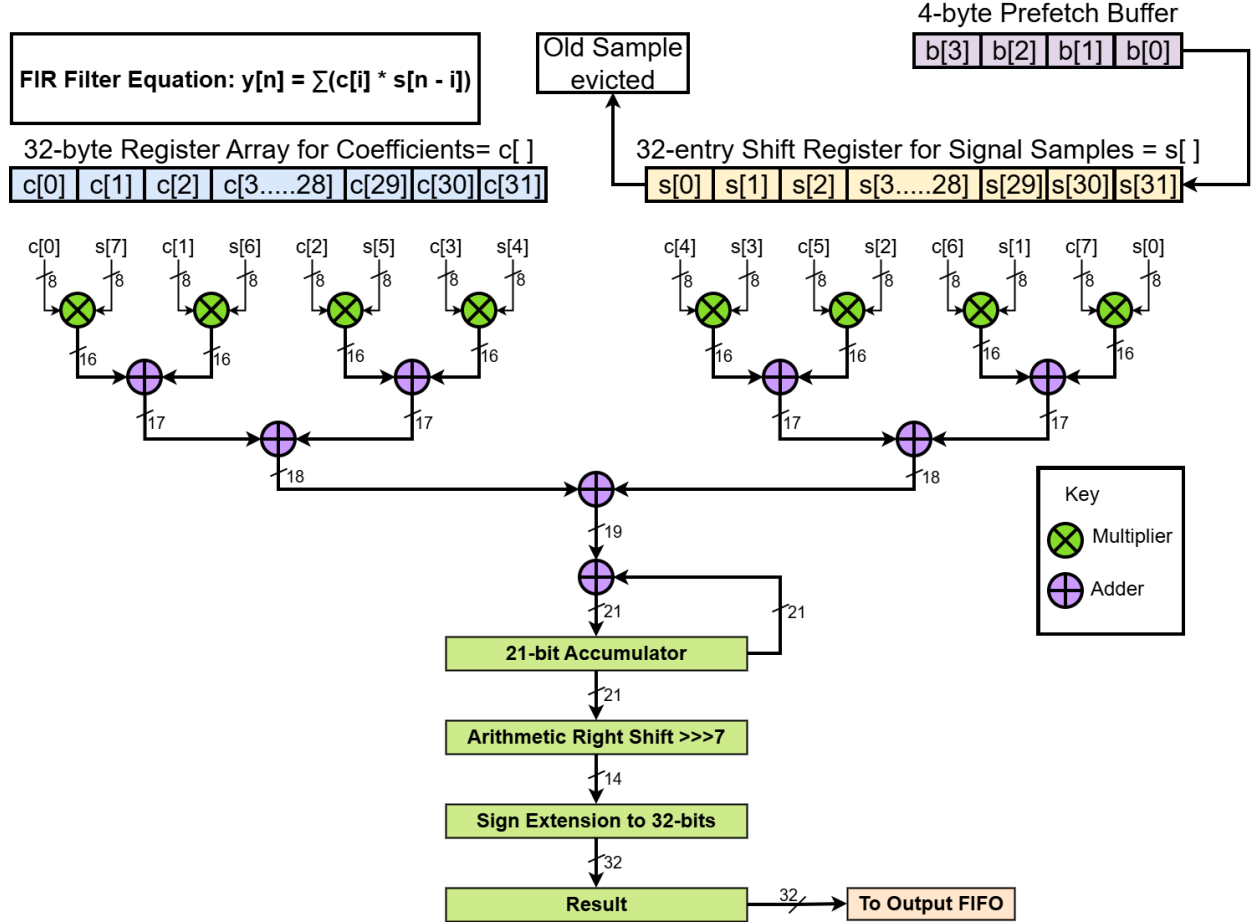


Figure 3: FIR Filter Accelerator datapath for the 8 MAC configuration (NumTaps = 32, NumMacs = 8)
 Note: Entries c[3] - c[28] and s[3] - s[28] are omitted from the diagram for clarity; all entries within the shift register and register array follow the same structure.

2.4 FIFO State Machine

Figure 4 shows the secondary writeback FSM which runs concurrently with the main FSM. It has two states: FIFO_IDLE and FIFO_WRITEBACK.

In FIFO_IDLE, the FSM monitors the output FIFO and transitions to FIFO_WRITEBACK as soon as the FIFO is non-empty.

In FIFO_WRITEBACK, the FSM drives write transactions to SRAM Bank 3 via the MGR OBI port, popping one result from the FIFO per granted transaction and incrementing the output address by 4 bytes after each write. To avoid bus conflicts on the shared MGR OBI port, writeback is paused whenever the `aux_refill_pending` flag is set or the main FSM is in READ_DATA state, ensuring read and write transactions never contend for the bus simultaneously. Once the FIFO is empty, the FSM returns to FIFO_IDLE.

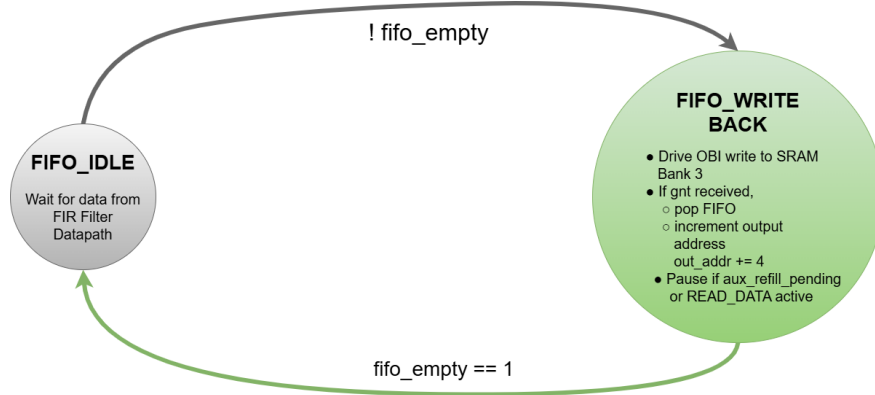


Figure 4: State diagram of the concurrent Output FIFO FSM

2.5 FIFO Datapath

Since the accelerator has only a single OBI master port, it cannot simultaneously issue a read instruction and write a result to SRAM. Stalling computation each time a write instruction is waiting would serialize these two operations and reduce throughput. We introduced a small Output FIFO buffer as shown in Figure 5, between the datapath and the writeback logic to decouple them. As each output sample is produced, it is immediately pushed into the FIFO, regardless of bus availability. We then drain the FIFO to the SRAM Bank 3 independently, yielding the bus only when a read instruction is needed. This improves overall throughput by overlapping computation with memory writeback.

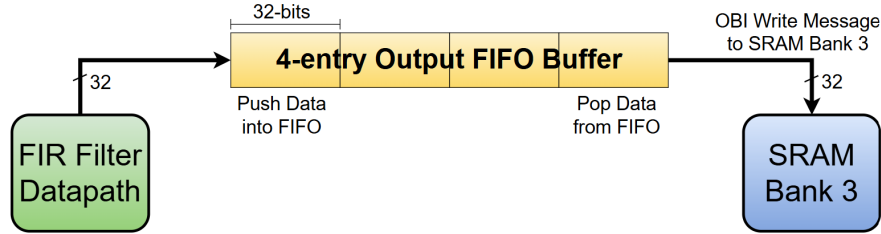


Figure 5: Output FIFO buffer interfacing the FIR filter datapath and SRAM Bank 3

2.6 Memory-Mapped Registers

Table 2 summarizes the memory-mapped registers exposed by the FIR accelerator at the base address 0x20000400. To program the accelerator, the CPU first writes all 32 filter coefficients to the coefficient registers at offsets 0x10 through 0x8C. It then writes the total sample count to offset 0x08, and finally asserts the start bit at offset 0x00. Once started, the CPU polls the done flag at offset 0x04 until it reads a non-zero value, signaling completion. If the provided sample count is outside the valid range of 36 to 540, the error flag at offset 0x0C is set, and no computation is started. The done flag is automatically cleared when a new start bit is asserted at the beginning of the next run.

Offset	Address	Access	Description
0x00	0x20000400	Write	Start computation (write 1 to trigger)
0x04	0x20000404	Read	Computation done flag (reads 1 when complete)
0x08	0x20000408	Write	Total sample count (16-bit, valid range 36 - 540)
0x0C	0x2000040C	Read	Invalid length flag (reads 1 on error)
0x10 - 0x8C	0x20000410 - 0x2000048C	Write	Filter coefficients c[0] - c[31]

Table 2: FIR Accelerator Memory-Mapped Registers

2.7 Software Driver and Accelerator Programming

The software driver programs and controls the FIR Filter accelerator through the following sequence. First, the raw integer coefficients are normalized to Q0.7 fixed-point by dividing each coefficient by the sum of all coefficients and scaling by 128, ensuring the filter has unity DC gain. The 32 normalized coefficients are then written to the Memory-Mapped registers at offsets 0x10 to 0x8C.

Next, the input signal samples are packed four per 32-bit word in little-endian byte order and written sequentially to SRAM Bank 2 starting at address 0x10001000. The total sample count is written to offset 0x08, after which the start bit at offset 0x00 is asserted. The driver then polls the done register at offset 0x04 until it reads 0x1, indicating computation is complete. Upon completion, the filtered results are read back from SRAM Bank 3, starting at 0x10001800, one 32-bit result per output sample, to print for debugging purposes.

3 Backend Implementation

The backend flow was implemented using Yosys for synthesis, OpenROAD for physical design, KLayout for Design Rule Checks (DRC), and Calibre for Layout-Versus-Schematic (LVS). The reference flow scripts were used as a baseline, with targeted modifications to support the FIR accelerator integration. The final design meets a frequency of 80 MHz and passes all setup, hold, DRC and LVS checks.

3.1 Floorplanning and SRAM Macro Placement

The baseline Croc SoC includes two SRAM banks serving as combined CPU instruction and data memory. To support the FIR accelerator, two additional 512x32-bit SRAM banks were added by setting `NumSramBanks` to 4: Bank 2 for input signal samples and Bank 3 for computed output results. The `instances.tcl` file was updated to map the new banks to their respective `RM_IHPSG13_1P_512x32` macro instances.

For macro placement, Bank 2 and Bank 0 were placed at the top-left and top-right respectively, with Banks 3 and 1 at the bottom-left and bottom-right respectively. Physical coordinates were set with a 20µm padding from the core boundary and a 10µm halo around each macro to prevent standard cell encroachment on the SRAM power ring.

3.2 Placement and Design Rule Check Fixes

Post-routing DRC checks revealed Metall spacing violations in the Croc SoC, specifically within the JTAG CDC path (`i_cdc.reset.ctrlr_half_b/i_state_transition_cdc_src`). These violations arose from insufficient spacing between adjacent standard cells at the M1 layer. Attempting to fix these manually on KLayout by moving these standard cells introduced thousands of new violations elsewhere.

Instead, a targeted placement padding approach was used. Using `get_cells -of_objects [get_nets ...]` command in OpenROAD, the violating cell instances were identified and a `set_placement_padding` constraint with `-left 1` and `-right 1` was applied to them specifically in `02_placement.tcl`, forcing OpenROAD's placer to introduce sufficient spacing. The entire backend flow was rerun and the M1 spacing violations were resolved without introducing new violations elsewhere.

3.3 Design Rule Violations

Post-routing analysis revealed 71 max slew violations, 210 max capacitance violations and 136 max fanout violations. Max slew and capacitance violations remained constant from the placement stage onwards, while max fanout violations increased from 0 after placement to 133 post-CTS, and 136 post-routing due to clock buffer insertion during clock tree synthesis.

Using `report_check_types -violators` in OpenROAD, all violations were traced to the Croc baseline infrastructure:

- Max Slew and Capacitance violations occur on the GPIO Pads.
- Max fanout violations are concentrated on the clock distribution nets introduced during CTS.

No violations originate from the FIR accelerator or user domain, confirming that the accelerator integration does not negatively impact the design’s electrical characteristics. All setup and hold timing violations were resolved to zero post-route, confirming functional correctness is unaffected.

3.4 Area Analysis

Table 3 presents the post-routing area summary of the complete Croc SoC. The total die-area is 4.63 mm^2 of which 2.20 mm^2 constitutes the core area available for standard cells and macro placement. Core utilization of 36.2% indicates that the design has sufficient routing headroom, which contributed to successful DRC closure with minimal violations. The FIR accelerator occupies $94,646 \text{ } \mu\text{m}^2$ of user domain area, representing only 4.3% of the total core area, confirming that the accelerator integrates into the Croc SoC with minimal area overhead.

Metric	Value
Die Area	$4,629,056.0 \text{ } \mu\text{m}^2$ ($\approx 4.63 \text{ mm}^2$)
Core Area	$2,200,258.4 \text{ } \mu\text{m}^2$ ($\approx 2.20 \text{ mm}^2$)
Total Cell Area	$2,439,784.7 \text{ } \mu\text{m}^2$ ($\approx 2.44 \text{ mm}^2$)
Total Active Area (std cells only)	$797,064.7 \text{ } \mu\text{m}^2$ ($\approx 0.80 \text{ mm}^2$)

Table 3: Post-routing design area summary of the Croc SoC with FIR accelerator integration.

Table 4 presents the hierarchical area breakdown of the key blocks within the Croc SoC. The four SRAM banks collectively occupy $321,880 \text{ } \mu\text{m}^2$ (14.6% of core area), making them the largest contributor, as expected for a memory-intensive design. The CVE2 core pipeline occupies $144,315 \text{ } \mu\text{m}^2$ (6.6%), with the register file and control/status registers accounting for the majority of this. The FIR accelerator user domain at $94,646 \text{ } \mu\text{m}^2$ (4.3%) is comparable in size to the debug module and JTAG infrastructure combined ($76,178 \text{ } \mu\text{m}^2$, 3.5%), confirming that the accelerator adds meaningful computational capability with a modest and well-justified area footprint.

Block	Area (μm^2)	% of Core Area
i_user (FIR Accelerator)	94,646.40	4.30
4× SRAM banks	321,879.80	14.63
i_core_wrap (CVE2 pipeline)	144,315.00	6.56
i_dm_top (Debug module)	48,915.00	2.22
i_dmi_jtag (JTAG/DMI)	27,263.00	1.24
Peripherals (i_uart, i_gpio, i_obi.timer)	53,196.70	2.42
Infrastructure / Support (i_clint, i_soc_ctrl, i_bootrom)	17,086.90	0.78

Table 4: Hierarchical area breakdown of key blocks in the post-routing Croc SoC layout

3.5 DRC and LVS Checks

Following all backend fixes, the final KLayout DRC check reported 512 Metal1 (M1) pin off-grid violations, all of which originated from the known issues in the IHP-provided SRAM macro and IO pad GDS files, where M1 pin shapes are not aligned to the 5nm manufacturing grid. As confirmed by the course’s teaching assistants, they do not represent real manufacturing concerns. Two Metal1 spacing violations were identified during routing and were resolved through targeted placement padding in `02_placement.tcl` as described in Section 3.2, resulting in zero DRC violations in the final GDS. LVS verification using Calibre, comparing the final `croc.gds` against the `croc.chip.SPICE` netlist returned no mismatches, confirming that the physical layout is electrically consistent with the post-route netlist.

In summary, the final design achieves no DRC violations except for the M1 pin off-grid violations, passes LVS, and meets the 80 MHz target with zero setup or hold violations.

4 Testing and Verification

4.1 System Constraints and Limitations

4.1.1 Design Constraints

1. The FIR Accelerator supports a minimum signal length of 36 samples and a maximum signal length of 540 samples.
2. The theoretical maximum signal length of 543 input samples is limited by the capacity of SRAM Bank 3 which can store up to 512 32-bit output words ($543 - 32 + 1 = 512$ outputs).
3. We conservatively limit the maximum to 540 samples rather than the theoretical maximum of 543, providing a small safety margin below the SRAM capacity limit.
4. Outside this range (36 to 540 samples), the `invalid_len` flag is asserted and computation is aborted.

4.1.2 Batch Processing Limitation

1. Since the Croc SoC platform lacks ADC/DAC peripherals for real-time ingestion of signal samples, our FIR accelerator follows a blocked FIR implementation.
2. In our current implementation, the signal samples must be pre-loaded into SRAM Bank 2 by the CPU before computation begins. The reported performance figures include the cycle counts and power consumed for this CPU-driven sample pre-loading overhead to SRAM Bank 2.
3. A streaming FIR accelerator implementation would be feasible with minimal RTL modifications such as replacing the batch SRAM interface with a FIFO-based input buffer, fed by an external ADC or UART peripherals, while keeping the MAC datapath unchanged.

4.2 Functional Verification

The FIR accelerator was verified in two stages: first validating the MAC datapath correctness in isolation, then validating the full accelerator’s behaviour including the FIFO and memory interfaces with varying signal lengths, coefficient values and signal sample values.

Golden Model: All hardware outputs were compared against a Python-based golden model with identical Q0.7 normalized coefficients and signed 8-bit signal samples.

4.2.1 Testing the Parameterized MAC Datapath

1. Basic functional tests were performed with signal lengths of 36, 64, 65, 371, 512, and 540 samples.
2. The test involving minimum valid signal length of 36 samples, with 32 samples filling the shift register and 4 samples in the auxiliary buffer requires no additional SRAM reads during computation.
3. This enabled isolated verification of the parameterized MAC datapath across all NumMacs configurations (1, 2, 4, 8, 16) without pausing the FIFO writeback or using the Prefetch Buffer refill logic.

4.2.2 Testing the System - Parameterized MAC Datapath + FIFO

1. For signal lengths beyond 36 samples, the accelerator must periodically pause result writeback to SRAM Bank 3 to fetch the next 4 samples from SRAM Bank 2 and reload the Auxiliary Buffer.
2. During this one-cycle pause, results accumulate in the 4-entry FIFO which then continues to drain once the read operation completes.
3. Tests with 64, 371, 512 and 540 samples exercised this interleaving between periodic reads and FIFO-buffered writes.

All results matched the Python golden model exactly. Correct results in all cases confirm proper handshaking between the FIFO, the MAC datapath and SRAM Banks 2 & 3.

4.2.3 Coefficient Edge Cases

1. Coefficient edge cases were used to stress-test the Q0.7 normalization logic in the software library.
2. Tests using all-extreme coefficient values (all coefficients = -128 or 127) also produced correct results without overflow or saturation errors.
3. Testing with all-zero coefficients revealed a division-by-zero scenario during normalization, since the coefficient sum appears in the denominator. This was resolved by adding a software guard, where if all coefficients are zero, normalization is skipped and the raw zero values are passed directly to the accelerator since zero representation is the same in Q8.0 and Q0.7.

4.2.4 Signal Sample Edge Cases

1. An all-zero signal confirmed that the output is identically zero regardless of coefficient values. Any non-zero output would indicate a buffer initialization error.
2. Stress tests using signals with all samples equal to 127 and all coefficients equal to 127 verified that the accumulator's 21-bit width prevents overflow across the full coefficient range.
3. Alternating +127/-128 signal samples verified the sign-handling correctness and subtraction behavior.

5 Performance Analysis

5.1 Accelerator Speedup Analysis

Table 5 compares the cycle counts and speed up for three implementations running a 32-tap FIR filter on 512 samples: Pure software implementation without a hardware multiplier, Software implementation with the CVE2 hardware multiplier enabled (with the RV32IM extension), and Implementation with the Hardware Accelerator with 8 MACs enabled.

Implementation	Cycles	Speedup vs SW Baseline	Speedup vs SW (with RV32IM)
SW Baseline	695586	1×	---
SW (with RV32IM)	331602	2.1×	1×
HW Accelerator (8 MACs)	7968	90.5×	41.6×

Table 5: Cycle count comparison across implementations

The software baseline without hardware multiplier requires 695586 cycles, as each MAC operation is emulated via shift-and-add operations. Enabling the hardware multiplier reduces this to 331602 cycles with a 2.1x speedup, demonstrating that the CVE2 core benefits meaningfully from the RV32IM extension for DSP workloads.

The hardware accelerator achieves 7968 cycles, yielding a 90.5x speed up over the software baseline and a 41.6x speed up over the hardware multiplier based implementation. This dramatic improvement is attributed to the parallel MAC datapath which operates autonomously via the MGR OBI port, fetching signal samples and writing results directly to SRAM without CPU involvement after the initial configuration.

5.2 Performance-Area Tradeoff

Table 6 presents the cycle counts, throughput, and user domain area for each NumMacs configuration (1, 2, 4, 8, 16), with the corresponding performance-area curve shown in Figure 6. User domain area was chosen as the area metric rather than the total chip area, as the Croc baseline (CPU, JTAG, IO pads, SRAM macros)

Number of MACs	Number of Cycles (#)	Throughput (Outputs/Cycle)	User Domain Area (μm^2)
1	22218	0.02165	58265.626
2	14073	0.03418	61271.482
4	10003	0.04809	72250.618
8	7968	0.06037	94646.362
16	7458	0.06449	133996.262

Table 6: Performance and area for each NumMacs configuration

occupies a largely fixed area regardless of the NumMacs configuration, and including it would obscure the actual cost of the accelerator design decision.

The plot in Figure 6 reveals a clear Pareto knee at 8 MACs.

- Doubling from 1 to 2 MACs yields a 58% throughput improvement with only a 5% area increase and similar gains are observed up to 8 MACs.
- Moving from 1 to 4 MACs yields a 122% throughput improvement with only a 24% area increase.
- Beyond 8 MACs however, the throughput gain diminishes significantly. Moving from 8 to 16 MACs increases the area by 41% while delivering only a 6.8% throughput improvement.

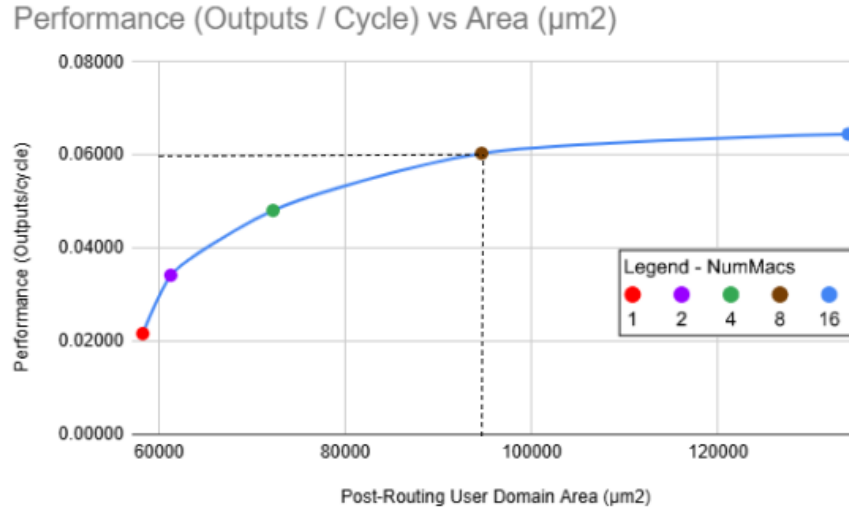


Figure 6: Performance-area tradeoff across NumMacs configurations, with the Pareto knee at 8 MACs

This diminishing return confirms 8 MACs as the Pareto-optimal configuration, offering the best balance between performance and silicon area, and justifies its selection as the final configuration.

5.3 Power Efficiency Analysis

Table 7 presents the stimuli-based power estimates and power efficiency (outputs/cycle/W) across all five NumMacs configurations with the corresponding power efficiency curve shown in Figure 7. Power was estimated using VCD-based activity annotation in OpenROAD at the tt process corner, using an identical 64-sample test signal across all NumMacs configurations, ensuring a fair and consistent comparison.

Total system power remains nearly constant across all configurations (21.4 - 21.6 mW), as it is dominated by clock distribution and SRAM Macros, both of which are independent of the NumMacs parameter. Consequently, power efficiency improves with NumMacs purely due to throughput gains rather than power reduction. However, diminishing returns are evident beyond 8 MACs. The marginal power efficiency gain from 4 to 8

MACs is 0.561 outputs/cycle/W, while the gain from 8 to 16 MACs drops to only 0.206 outputs/cycle/W, a 63% reduction in incremental benefit.

Number of MACs	Number of Cycles (#)	Throughput (Outputs/Cycle)	Total Power (W)	Power Efficiency (Outputs/cycle/W)
1	22218	0.02165	0.0216	1.002
2	14073	0.03418	0.0215	1.590
4	10003	0.04809	0.0214	2.247
8	7968	0.06037	0.0215	2.808
16	7458	0.06449	0.0214	3.014

Table 7: Power efficiency across NumMacs configurations

It should be noted that power efficiency increases monotonically across all configurations. 16 MACs remains the most power-efficient point, however when considered jointly with the 41% user domain area penalty, the 8 MAC configuration emerges as the Pareto-optimal choice.

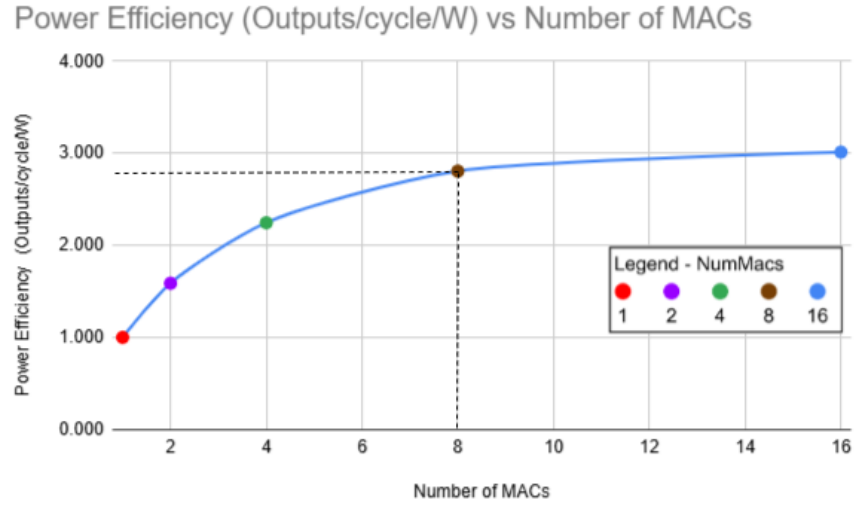


Figure 7: Performance-area tradeoff across NumMacs configurations, with the Pareto knee at 8 MACs

6 Conclusion

In this project, we successfully extended the Croc SoC with a parameterizable 32-tap FIR filter accelerator, demonstrating significant performance improvements over software execution on the CVE2 core. The accelerator follows a blocked FIR architecture with a shift-register-based filter window, a prefetch buffer for concurrent SRAM reads, and a decoupled FIFO writeback mechanism to maximize throughput on a single OBI master port.

We systematically evaluated five configurations (1, 2, 4, 8, and 16 MAC units) across performance, area, and power dimensions. The 8 MAC configuration was identified as the Pareto-optimal design point, achieving a $90.5\times$ speedup over the software baseline and a $41.6\times$ speedup over the RV32IM-accelerated software implementation, while limiting user domain area growth to 62% compared to the 1 MAC baseline. Power analysis confirmed that total system power remains nearly constant (21.5 mW) across all configurations, dominated by clock distribution and SRAM macros rather than the accelerator logic itself.

The backend implementation was carried out using Yosys and OpenROAD, targeting the IHP SG13G2 process at 80MHz. Timing closure was achieved with WNS = 0ns, and all setup and hold violations were resolved. DRC and LVS checks passed successfully, with only known PDK-related M1 pin off-grid violations in the SRAM macros. The final design culminates in a fully placed-and-routed GDSII layout ready for submission.